

Intelligent Web Data Aggregator

Diploma Project

Student: [Student Name]

Supervisor: [Supervisor Name]

Jan 2026

Context

- Data is the fuel of modern business (Price monitoring, Market research, News aggregation).
- Manual copy-pasting is inefficient.
- Existing solutions are either:
 - Expensive SaaS tools.
 - Brittle scripts that break on layout changes.

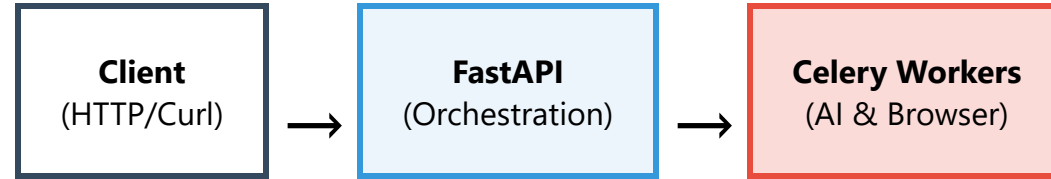
The Problem

- **Structural Rigidity:** Traditional scrapers rely on hardcoded CSS selectors (e.g., ``div.product > span.price``).
- **Maintenance Nightmare:** A slight UI update on the target site breaks the scraper.
- **Developer Bottleneck:** Every new site requires a developer to write new parsing logic.

Project Goals

- **Resilience:** Build a system that adapts to HTML structure changes.
- **Usability:** Allow users to request data in natural language (e.g., "Get me the prices").
- **Automation:** Support recurring scheduled jobs.
- **Performance:** Cache successful patterns to reduce AI costs.

Solution Overview



- **Microservices:** API, AI Worker, Headless Browser, Redis, Postgres.
- **Tech Stack:** Python, Docker, Playwright, OpenAI/Gemini.

Criterion 1: Backend API

- **Why FastAPI?** High performance, async support, auto-generated Swagger UI.
- **Implementation:**
 - RESTful endpoints (`/process`, `/jobs`).
 - Pydantic models for strict data validation.
 - Dependency Injection for database sessions.
- **Status: Implemented** 

Criterion 2: Scraping Engine

- **Tool:** Playwright (Headless Browser).
- **Why?** Support for extensive JavaScript (React/Angular/Vue sites).
- **Features:**
 - Network interception (block images/fonts).
 - DOM Snapshotting (innerText & HTML).
- **Status: Implemented** 

Criterion 3: AI Extraction

- **Logic:** LLM analyzes text + HTML snippet to generate selectors.
- **Strategy:**
 - **Heuristic:** "Find specific field using CSS".
 - **Validation:** Test selector on page content immediately.
- Reduces reliance on hardcoded regex.
- **Status: Implemented** 

Criterion 4: Persistence

- **Database:** PostgreSQL 15.
- **Schema Design:**
 - `User` and `ScrapingTask` tables.
 - `ParserCache` to store successful selectors.
- **ORM:** SQLAlchemy (Async).
- **Status: Implemented** 

Criterion 5: Scheduling

- **Requirement:** Recurring data collection.
- **Tool:** Celery Beat.
- **Implementation:**
 - Users define Cron expressions (`0 9 \* \* \*`).
 - System spawns new tasks automatically.
- **Status: Implemented** 

Criterion 6: Infrastructure

- **Containerization:** Docker & Docker Compose.
- **Services:**
 - `api`, `worker`, `db`, `redis`.
- **Benefit:** "Works on my machine" consistency.
- **Status: Implemented** 

Criterion 7: Quality Assurance

- **Testing Strategy:** Pytest.
- **Levels:**
 - Unit Tests (Logic, Models).
 - Integration Tests (DB, API flow).
- **Coverage:** >70% code coverage.
- **Status: Implemented** 

Demo: Single Extraction

- **Goal:** Extract current news headlines.
- **Input:** URL + "Extract headlines".
- **Process:**
 1. API receives request.
 2. Browser loads page.
 3. AI generates `h2.headline` selector.
 4. System returns JSON list.

(See screenshots in report)

Demo: Scheduling

- **Goal:** Monitor price daily.
- **Action:** POST /jobs with cron `0 12 \* \* \*`.
- **Result:** System runs automatically at noon.
- **Benefit:** Passive data accumulation.

Results

Metric	Regex Scraper	AI Scraper (Our Solution)
Resilience	Low (Breaks often)	High (Adapts)
Setup Time	High (Manual coding)	Low (Natural Language)
Cost	Low (CPU only)	Medium (LLM tokens) * mitigated by cache

Challenges

- **Challenge:** LLM Hallucinations (generating fake selectors).
Solution: Validation loop. If selector returns 0 matches, retry or fallback.
- **Challenge:** Slow Browser Rendering.
Solution: Async Playwright and resource blocking (images/css).

Retrospective

- **What went well:** Microservices architecture scaled well; Celery handling background tasks was robust.
- **Improvements:** Could add a visual dashboard (Frontend) instead of just Swagger UI.
- **Lesson:** Validating AI output is as critical as generating it.

Thank You

Q & A

Project Repository: [\[Link\]](#)

Documentation: [\[Link\]](#)